A Snowflake-Centered Personalized Research Assistant with RAG, Knowledge Graphs, and Evaluation

CS 5542 - Big Data Analytics and Applications

Project Proposal

Team Members

- Blake Simpson
- Kenneth Kakie
- Rohan Ashraf Hashmi

Github:

https://github.com/BigDataAnalytics-CS5542/snowflake-research-assistant

Abstract

The rapid growth of scientific literature has made it increasingly difficult for researchers to efficiently search, synthesize, and validate information across large personal collections of papers. While large language models (LLMs) offer powerful natural language interfaces, they suffer from hallucinations and lack transparency when operating without grounding in source material.

In this project, we propose a Snowflake-centered Personalized Research Assistant that combines Retrieval-Augmented Generation (RAG) with knowledge-graph–enhanced retrieval to provide grounded, explainable, and structured answers over user-uploaded research papers. Our system allows users to upload documents, ask natural language questions, receive cited responses with confidence scores, and explore relationships between papers, concepts, and methods through graph-based reasoning.

To ensure research-grade rigor, we incorporate an evaluation pipeline comparing standard RAG against fine-tuned and graph-enhanced retrieval strategies, measuring accuracy, citation correctness, latency, and scalability. The final deliverable is a working web-based prototype backed by a reproducible Snowflake data pipeline, demonstrating how structured intelligence and GenAI can be combined into a deployable research assistant rather than a simple LLM demo.

1. Problem Statement and Objectives

1.1 Problem Statement

Researchers today face an overwhelming volume of scientific publications across disciplines, conferences, and journals. Even within a single research area, it is common for individuals to manage hundreds of PDFs containing overlapping concepts, methods, and results. Traditional keyword search tools fail to capture semantic relationships across documents, while LLM-based chat systems often generate unverified or hallucinated answers when operating without strong grounding.

Moreover, existing research tools rarely provide visibility into why an answer was produced, how confident the system is, or how information from multiple papers is connected. This lack of transparency limits trust and reduces the usefulness of AI systems for serious academic and industrial research.

1.2 Target Users

- Graduate students conducting literature reviews
- Academic researchers synthesizing results across papers
- Industry R&D teams maintaining internal research corpora

1.3 Objectives

The primary objectives of this project are to:

- Build a personalized research assistant over user-uploaded scientific documents
- Ground all generated responses in source text using Retrieval-Augmented Generation (RAG)
- Enhance retrieval and reasoning using knowledge-graph–based structure
- Provide citations and confidence estimates for generated answers
- Evaluate different retrieval strategies (RAG vs. fine-tuned vs. graph-enhanced)
- Design a Snowflake-centered, reproducible, and scalable system architecture

2. Project Evolution and Innovation Layer

Our pre-proposal introduced two separate project ideas: a Personal Research Assistant and a Knowledge Graph Research Explorer. Based on instructor feedback emphasizing the need for greater technical ambition, we integrate these ideas into a single unified system.

Rather than treating knowledge graphs as a standalone product, this project incorporates knowledge-graph extraction as an internal intelligence layer within the research assistant. Entities such as papers, authors, methods, datasets, and citations are automatically extracted and modeled as a lightweight graph. This structure enables advanced retrieval strategies, cross-document reasoning, and optional graph-based exploration, while keeping the primary user interaction centered on natural language question answering.

This integration directly addresses the instructor's recommendation to expand beyond basic RAG by introducing graph-based queries and structured retrieval, elevating the system from a standard class assignment to a research-oriented prototype.

3. System Overview

At a high level, the proposed system consists of five major components:

1. **Document Ingestion Pipeline**

   User-uploaded PDFs are processed, chunked, embedded, and stored using Snowflake stages and tables.

2. **Vector and Graph Storage**

   Text embeddings support semantic retrieval, while extracted entities and relationships form a lightweight knowledge graph.

3. **Retrieval Layer**

   Relevant passages and subgraphs are retrieved based on the user query.

4. **Generation Layer**

   An LLM generates grounded answers using retrieved context, citations, and confidence estimation.

5. **Evaluation and Monitoring**

   Retrieval strategies are compared using offline benchmarks and runtime metrics.

This architecture enables modular experimentation while remaining deployable in a production-like environment.

4. Related Work (NeurIPS 2025)

4.1 GFM-RAG: Graph Foundation Model for Retrieval Augmented Generation

Paper link: https://neurips.cc/virtual/2025/poster/120321
Code link: https://github.com/RManLuo/gfm-rag

Summary:

GFM-RAG introduces a graph foundation model that builds a knowledge graph index from documents to improve retrieval performance for RAG tasks. Instead of only using standard vector search, it leverages graph neural networks to capture complex relationships between pieces of knowledge and obtain more relevant context for an LLM to generate answers. This method demonstrates better handling of multi-source reasoning by explicitly modeling inter-entity relations.

Project integration:

Our proposal's KG layer can leverage concepts from GFM-RAG's graph indexing to structure paper metadata and entity connections (e.g., authors, methods, citations) inside Snowflake. The graph index could inform advanced retrieval queries that complement vector search, improving answer quality and explainability in our system.

4.2 Can Knowledge-Graph-based Retrieval Augmented Generation Really Retrieve What You Need?

Paper link: https://neurips.cc/virtual/2025/poster/115922

Summary:

This work examines the effectiveness of knowledge-graph-based RAG methods and proposes GraphFlow, a framework that retrieves accurate and diverse knowledge from text-rich knowledge graphs for real queries. Unlike traditional RAG relying solely on vectors, this method emphasizes structured retrieval over graph embeddings and dynamic retrieval paths tailored to query needs.

Project integration:

GraphFlow's structured retrieval ideas directly inform our advanced retrieval layer. In the research assistant, we can incorporate GraphFlow-inspired approaches to improve how we fetch context for a query from a constructed graph of research entities, then fuse that with vector-based context before generation.

4.3 Chain-of-Retrieval Augmented Generation

Paper link: https://neurips.cc/virtual/2025/poster/116740

Summary:

This paper proposes a multi-step RAG framework that performs sequential retrieval and reasoning before final generation. Instead of a single retrieval step, it uses iterative retrieval to refine context for complex queries, which significantly improves the relevance and completeness of the final generated answer.

Project integration:

Our research assistant will benefit from this paper's sequential retrieval strategy. Inside our pipeline, we can implement a similar multi-stage retrieval approach where we first retrieve broadly relevant chunks, optionally filter/refine with graph context, and then perform a second retrieval round before passing the refined context to the LLM for answer generation.

4.4 How These Papers Fit Our Snowflake Pipeline

Paper	Snowflake Integration	Purpose
GFM-RAG	Graph indexing stored in Snowflake tables + graph execution via a graph engine	Enhances retrieval with structural relationships
KG-RAG (GraphFlow)	Use Snowflake to query structured KG info and combine with semantic vectors	More accurate and diverse retrieval
Chain-of-Retrieval	Iterative retrieval workflows expressed in Snowpark jobs or stored procedures	Repeated refinement of context before generation

5. Data Sources & Snowflake Ingestion Plan

For our Snowflake-centered Personalized Research Assistant, we need datasets that give us real scientific papers, textual content to retrieve from, and potential QA benchmarking ability. Below are three high-value datasets that align with these needs, along with ingestion plans suitable for a Snowflake analytics + GenAI pipeline.

5.1 arXiv Scientific Papers Dataset

Dataset link: Kaggle – arXiv dataset and metadata of 1.7M+ scholarly papers
URL: https://www.kaggle.com/datasets/Cornell-University/arxiv

Description:

A richly featured corpus of more than 1.7 million research papers across STEM disciplines. This dataset includes metadata such as titles, abstracts, authors, categories, and full text (where available). It is a foundational resource for building retrieval and indexing systems over research literature.

Why it fits:

- Provides a large, open corpus for ingestion.
- Metadata supports entity extraction (authors, terms, keywords) useful for knowledge graph construction.
- Works as the primary document source for our RAG system.

Ingestion Plan (Snowflake):

1. Export the compressed dataset (CSV/Parquet) to a Snowflake stage (e.g., AWS S3).
2. Create external or internal Snowflake tables for:
    - Paper metadata (title, authors, categories)
    - Full text / abstracts (long text)
3. Use Snowpipe for any incremental updates if we extend with new papers.
4. Create views for semantic chunking pipelines.

This dataset forms the backbone of our RAG pipeline because the LLM will need to retrieve specific paper contexts and citations.

5.2 Scientific Papers (ArXiv + PubMed) from Hugging Face Datasets

Dataset link: Hugging Face – armanc/scientific_papers
URL: https://huggingface.co/datasets/armanc/scientific_papers

Description:

This dataset combines two large repositories - ArXiv and PubMed OpenAccess scientific articles - in structured JSON format. Each entry includes the full body of the paper, section boundaries, and abstract text.

Why it fits:

- Contains long-form structure (organized sections), which improves chunking quality for semantic search.
- Enables training or benchmarking of models on real long documents rather than short abstracts.

Ingestion Plan (Snowflake):

1. Download the Hugging Face dataset into cloud storage (S3 or GCS).
2. Load the data via Snowflake stages into:
    - papers_structured table with article, abstract, section_names
    - JSON columns for flexible schema evolution
3. Use Snowpark Python to preprocess:
    - split sections into chunks
    - extract embeddings per chunk
4. Populate a vector store in Snowflake or external vector DB integrated with Snowflake.

Because this dataset maintains individual section boundaries, it supports enhanced retrieval beyond coarse document chunks.

5.3 Academic Papers: Titles & Abstracts

Dataset link: Kaggle - 3M+ Academic Papers: Titles & Abstracts
URL: https://www.kaggle.com/datasets/beta3logic/3m-academic-papers-titles-and-abstracts

Description:

A broad collection of over 3 million research paper titles and abstracts compiled from multiple sources. While it does not always include full paper text, it provides a massive pool of abstracts and metadata that can be used for fast retrieval, indexing, and facet-based search.

Why it fits:

- Increases the breadth of our corpus significantly.
- Useful for coarse indexing and learning high-level semantic structures.
- Abstracts are frequently sufficient for first-stage retrieval in multi-stage RAG pipelines.

Ingestion Plan (Snowflake):

1. Load the dataset into Snowflake via staged files.
2. Create a table of abstracts with indexing on title, year, and categories.
3. Generate embeddings for abstracts and titles to power quick semantic filtering.
4. Use abstracts first during retrieval to narrow down candidate documents.

This dataset serves as a fast filtering layer before deeper retrieval from full-text sources (arXiv and Hugging Face).

Snowflake Pipeline Sketch (How It All Connects)

Here's how these datasets fit into a reproducible Snowflake workflow:

1. **Data Ingestion**
    - Export datasets to cloud storage (S3 / GCS).
    - Use Snowflake stages + COPY INTO to load into structured tables.
2. **Processing & Semantic Prep**
    - Use Snowpark Python to:
        - Chunk documents (split by sections or fixed length)
        - Extract embeddings (per chunk and abstract)

    - Store embeddings in a vector structure (Snowflake vector support or external store)
3. **Knowledge Graph Layer**
    - Run entity extraction (authors, methods, keywords) on metadata + full text.
    - Populate entity tables (paper_entity, author_entity, term_entity).
    - Populate relationship tables (paper_cites, author_wrote, term_in_paper).
4. **Retrieval Layer**
    - Use vector search + graph queries to select candidate texts.
    - Combine results for the RAG model's context.
5. **Generation + Evaluation**
    - Generate answers using grounded context.
    - Compare RAG only vs. KG-augmented retrieval in evaluation metrics.

6. Methods, Technologies, and System Architecture

This section outlines the end-to-end system design of the Personalized Research Assistant and how Retrieval-Augmented Generation (RAG) is enhanced with knowledge-graph–based retrieval to improve grounding, explainability, and cross-document reasoning.

6.1 System Architecture Overview

The system is organized as a modular pipeline with four core components:

- **Data Layer (Snowflake-centered):** stores documents, chunks, embeddings, and extracted entities
- **Retrieval Layer:** combines vector-based semantic search with graph-based filtering
- **Generation Layer (LLM):** produces grounded answers with citations
- **Evaluation Layer:** logs answer quality and system performance metrics

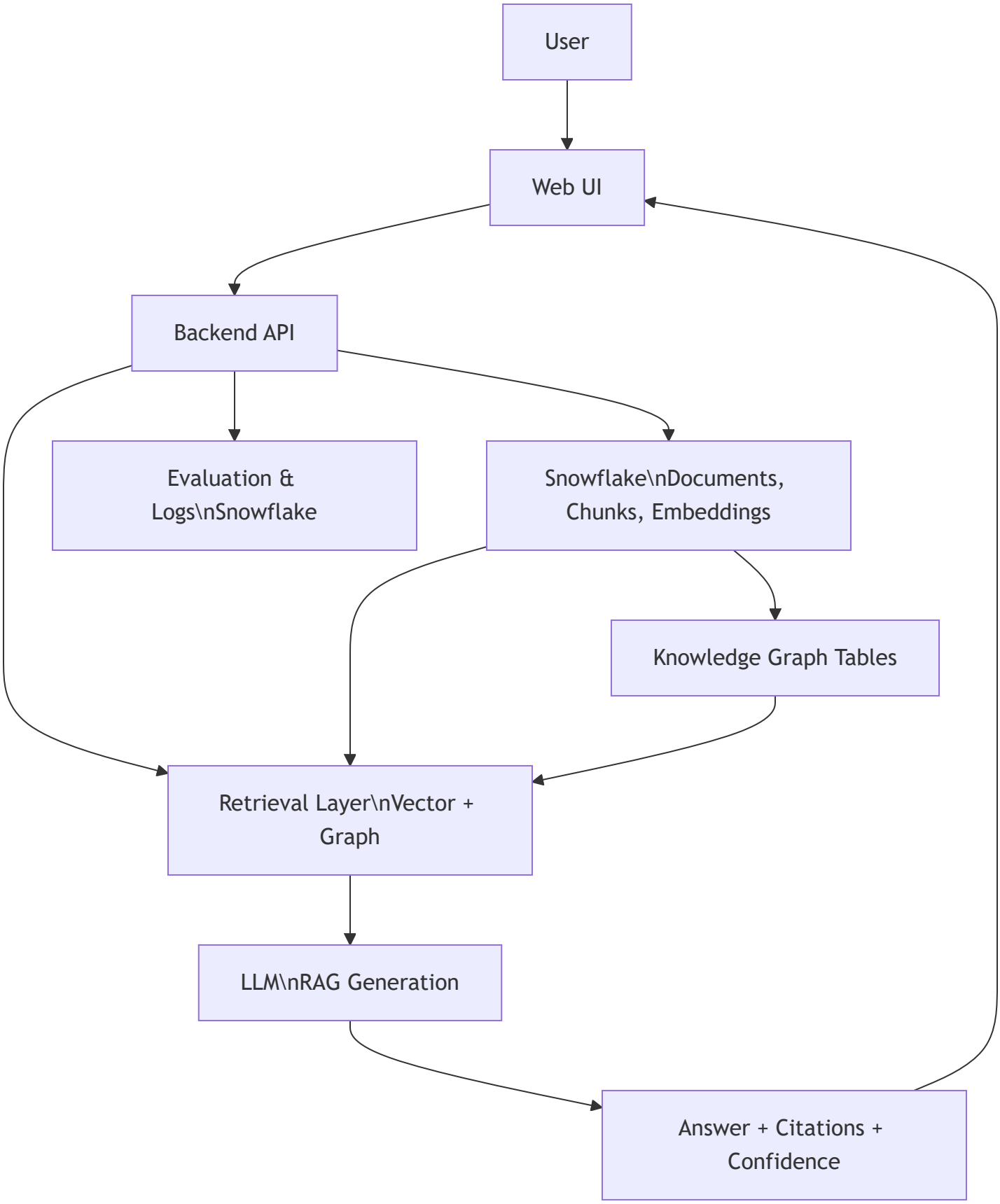

Figure 1 shows the high-level system architecture.

This modular design enables experimentation without reworking the entire system.

6.2 Document Processing and Retrieval

Uploaded documents are processed using **Snowpark Python**, where text is split into semantic chunks based on section or paragraph boundaries. Each chunk is stored with identifiers and source metadata, enabling accurate citation and traceability.

Chunks and abstracts are embedded using a pretrained model for scientific text. Retrieval follows a two-stage approach: an initial vector-based search to identify candidate passages, followed by re-ranking using metadata and optional graph-based constraints to balance precision and recall.

6.3 Knowledge Graph–Enhanced Retrieval

To address limitations of standard RAG, the system incorporates a lightweight knowledge graph extracted from user-uploaded papers. Entities such as papers, authors, methods, and concepts, along with relationships like citations and method usage, are stored as normalized Snowflake tables.

During retrieval, graph relationships are used to prioritize closely connected or authoritative papers, improving cross-document reasoning, consistency, and explainability while reducing hallucinations.

6.4 Generation and Evaluation Design

Answers are generated by an LLM constrained to retrieved context and returned with explicit citations and confidence indicators derived from retrieval consistency.

The system supports comparative evaluation across standard RAG, fine-tuned RAG, and graph-enhanced RAG using identical datasets and queries, enabling controlled analysis of how advanced retrieval strategies affect answer quality.

7. Infrastructure, Deployment, and Monitoring

To ensure the system functions as a research-grade prototype rather than a conceptual demo, we design the infrastructure with deployment realism, scalability, and observability in mind. The architecture mirrors production systems while remaining lightweight enough for academic development.

7.1 Deployment Architecture

The system follows a modular service-oriented design that separates data management, retrieval logic, and user interaction.

- **Backend Services**
  A lightweight API layer handles document upload, query submission, retrieval orchestration, and response assembly. This layer interfaces with Snowflake for data access and analytics and with the LLM provider for generation.
- **Snowflake Data Platform**
  Snowflake serves as the central system of record for ingested papers, chunked text, embeddings, knowledge graph entities, and evaluation metrics. Snowpark jobs support preprocessing, embedding generation, and scheduled evaluation tasks.
- **Frontend Interface**
  A web-based UI enables users to upload papers, submit queries, and view grounded answers with citations and confidence indicators, with optional exploration of related papers or concepts.

This separation allows independent iteration on retrieval strategies, UI design, and evaluation logic without destabilizing the system.

7.2 Cloud and Runtime Environment

The system is designed to run on standard cloud infrastructure (e.g., AWS or GCP) while remaining cloud-agnostic. Backend services are containerized for reproducibility, APIs are stateless, and Snowflake maintains persistent state.

Snowflake role-based access control enforces user-scoped data isolation, ensuring that each user's documents and embeddings are accessible only within their assigned role. Evaluation and monitoring tables are exposed through read-only roles to support governance and reproducibility without risking data integrity.

7.4 Reproducibility, Scalability, and Extensibility

All code and documentation are maintained in a shared GitHub repository, with dataset versions, preprocessing steps, and experiment configurations explicitly documented. Snowflake tables store experiment identifiers alongside results to enable repeatable and auditable comparisons.

While the prototype targets a limited initial scale, the architecture supports growth through Snowflake's scalable storage and parallel retrieval and generation components. Future extensions may include

richer graph analytics, domain-specific fine-tuning, and enhanced visualization of research relationships.

8. Expected Outcomes and Evaluation Metrics

This section defines the concrete outcomes and evaluation criteria used to assess the success of the proposed Personalized Research Assistant. Rather than relying on qualitative impressions, we focus on measurable metrics aligned with accuracy, grounding, transparency, and scalability.

8.1 Expected System Outcomes

By the end of the project, we will deliver a functional research assistant prototype that enables users to:

- Upload scientific papers and build a personalized document corpus
- Ask natural language questions over their uploaded documents
- Receive grounded answers with:
  - explicit citations to source documents or sections
  - confidence indicators based on retrieval consistency
- Compare multiple retrieval strategies, including:
  - standard RAG
  - fine-tuned RAG
  - knowledge-graph–enhanced RAG

These outcomes emphasize explainability and trust, which are critical for real-world research use.

8.2 Live Demo Scope

The final demo will showcase:

- Document upload with automatic preprocessing and indexing
- Question answering with cited and confidence-scored responses
- Side-by-side comparison of standard RAG and graph-enhanced retrieval
- Visualization of retrieval metrics and citation coverage

This demo highlights how retrieval design choices impact answer quality.

8.3 Evaluation Metrics

Evaluation focuses on answer quality and system performance.

Answer Quality Metrics

- **Answer Accuracy** – alignment with reference answers where available
- **Citation Correctness** – whether cited passages support the answer
- **Context Coverage** – proportion of relevant documents retrieved

System Metrics

- **Retrieval Precision and Recall**
- **Latency** – end-to-end response time
- **Scalability** – performance as corpus size increases
- **Hallucination Rate** – answers lacking sufficient citation support

All metrics are logged in Snowflake to enable repeatable analysis.

8.4 Comparative Evaluation and Success Criteria

Using a fixed set of queries and documents, we will compare standard RAG, fine-tuned RAG, and graph-enhanced retrieval under identical conditions. This allows us to isolate the impact of structured retrieval on answer quality.

The project will be considered successful if:

- The system consistently produces grounded, cited answers
- Graph-enhanced retrieval improves citation quality and cross-document reasoning
- Evaluation results are reproducible across multiple runs

9. Reproducibility, Repository Structure, and Project Timeline

A core objective of this project is to ensure that all results, experiments, and system behaviors are reproducible and auditable. To support this, we emphasize disciplined repository organization, clear documentation, and a well-defined development timeline.

9.1 GitHub Repository Structure (repo link here)

All project artifacts will be maintained in a shared team GitHub repository accessible to all members. The repository is organized to support collaboration, transparency, and reproducibility.

Planned repository structure:

```
/README.md
/proposal
/docs
/data
/reproducibility
/backend
/frontend
/evaluation
```

- **README.md**
  Project overview, objectives, team members, system architecture diagram, dataset links, and related work references.
- **/proposal**
  Final proposal PDF and any submitted revisions.
- **/docs**
  System architecture diagrams and design notes.
- **/data**
  Dataset references and preprocessing scripts (raw data excluded).
- **/reproducibility**
  Environment setup instructions and experiment configuration templates.
- **/backend / /frontend**
  Application source code and API/UI components.
- **/evaluation**
  Benchmark queries, evaluation scripts, and metrics aggregation.

This structure ensures that another researcher or evaluator can reproduce the system without relying on undocumented assumptions.

9.2 Reproducibility Strategy

Reproducibility is supported at both the data and model levels through:

- **Versioned datasets**, with sources and preprocessing steps explicitly documented

- **Configuration-driven experiments**, where retrieval strategies and model parameters are logged per run
- **Logged results**, with retrieval metrics, outputs, and performance statistics stored in Snowflake tables
- **Deterministic evaluation where possible**, with fixed random seeds and documented sources of nondeterminism

These practices enable side-by-side comparison of experiments and traceability of performance changes.

9.3 Collaboration and Role Alignment

Team responsibilities align with the system's modular design:

- **Backend & Retrieval**: document ingestion, retrieval orchestration, and RAG integration
- **Data Infrastructure & MLOps**: Snowflake pipelines, Snowpark processing, and evaluation logging
- **Frontend & Visualization**: user interface for document upload, querying, and result presentation

Clear role delineation supports parallel development while maintaining system coherence.

9.4 Project Timeline and Milestones

The project will be executed in structured phases:

- **Phase 1 — Data & Ingestion Setup**
  Dataset finalization, Snowflake ingestion, preprocessing validation
- **Phase 2 — Core RAG System**
  Vector retrieval, LLM generation with citations, baseline evaluation
- **Phase 3 — Knowledge Graph Integration**
  Entity extraction, graph-enhanced retrieval, comparative analysis
- **Phase 4 — Evaluation & Refinement**
  Benchmark execution, metric analysis, system optimization
- **Phase 5 — Final Demo & Documentation**
  Live demo preparation, documentation finalization, reproducibility packaging

This phased approach ensures steady progress while keeping the system functional throughout development.

10. Conclusion

This project proposes a Snowflake-centered Personalized Research Assistant that addresses the growing challenge of navigating and synthesizing large collections of scientific literature. By combining Retrieval-Augmented Generation (RAG) with knowledge-graph–enhanced retrieval, the system moves beyond standard LLM-based question answering toward a more structured, explainable, and evaluable research tool.

Unlike generic chat-based assistants, the proposed system grounds all responses in user-provided documents, provides explicit citations and confidence indicators, and incorporates structured relationships between papers, authors, methods, and concepts. This design directly addresses known limitations of existing approaches and aligns with the instructor’s feedback to expand technical ambition through advanced retrieval and graph-based reasoning.

The Snowflake-centered architecture supports scalable data ingestion, semantic retrieval, graph analytics, and rigorous evaluation within a unified platform. By treating evaluation, monitoring, and reproducibility as first-class concerns, the project emphasizes research rigor while remaining practical and deployable.

Overall, this work demonstrates how structured intelligence, data engineering, and generative AI can be combined to build a trustworthy and extensible research assistant. The methods and infrastructure outlined here provide a strong foundation for future extensions such as domain-specific fine-tuning, richer graph analytics, and broader application to enterprise knowledge management systems.